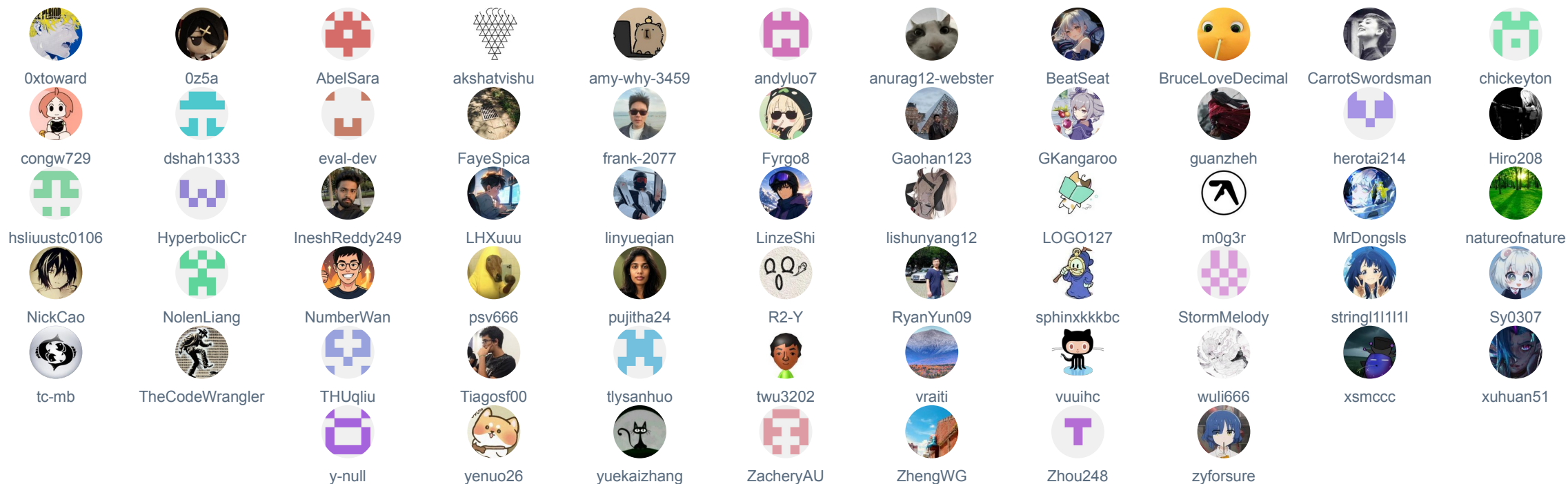


The Next Inflection Point in AI Interaction

How vLLM-Omni Enables Models to
Listen and Speak Simultaneously

Full duplex, built with the community

Thanks to 60+ contributors · 160+ related PRs merged



SELECTED MERGED PRs

Native duplex runtime · #3907	Unified duplex framework · #7413	OpenAI Realtime API · #7285	Qwen3-Omni integration · #7585
PersonaPlex unified duplex · #7695	Nemotron VoiceChat · #6089	AURA integration · #7633	Bounded output delivery · #7644

AS OF 07 OCT 2026

Keep the conversation flowing.

01 Concurrent input & output

Keep receiving speech and video while the assistant responds.

02 Streamed responses

Deliver text and audio incrementally, without waiting for a complete reply.

One live conversation

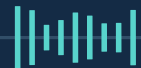
Camera



User speech



Assistant speech



barge-in

03 Adaptive interaction

Use new input to continue, adapt what comes next, or yield the floor.

Hard interruption: forced stop

Client / server control

Explicit cancel or configured VAD policy

Soft interruption: model-driven yielding*

The model uses ongoing input + context to decide when to stop.

*Depends on model capability.

How models handle live interaction

Web demo examples: model decisions, Server VAD and client controls shape the interaction.

MODEL	INTERACTION	WHAT HAPPENS
MiniCPM-o 4.5	Continuous speech and video input Model decides LISTEN / SPEAK	Listen or respond within the live session No Server VAD required by default
Qwen3-Omni	Server VAD detects user speech Microphone input stays active	Stop queued playback Interrupt the current response*
AURA	Hold to talk; release to submit new audio / video input	Holding stops local playback Server generation can continue

Applications can add explicit `response.cancel` for server-side cancellation.

MODEL YIELDING

The model switches to listening
LISTEN is not `response.cancel`

LOCAL PLAYBACK STOP

Stop what the user hears
Generation may continue

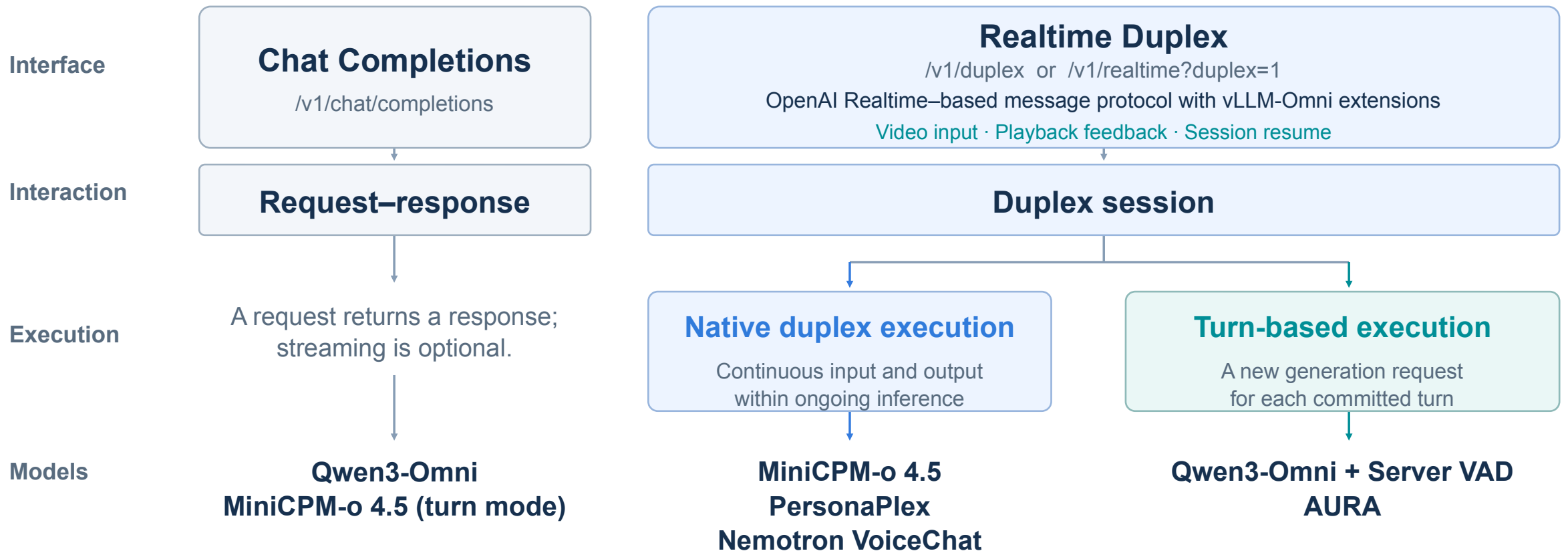
SERVER CANCELLATION

Cancel the current response
Client clears queued audio

* Server VAD interruption requires `interrupt_response`. Behavior depends on the model and selected demo profile.

Interfaces, interaction modes and model execution

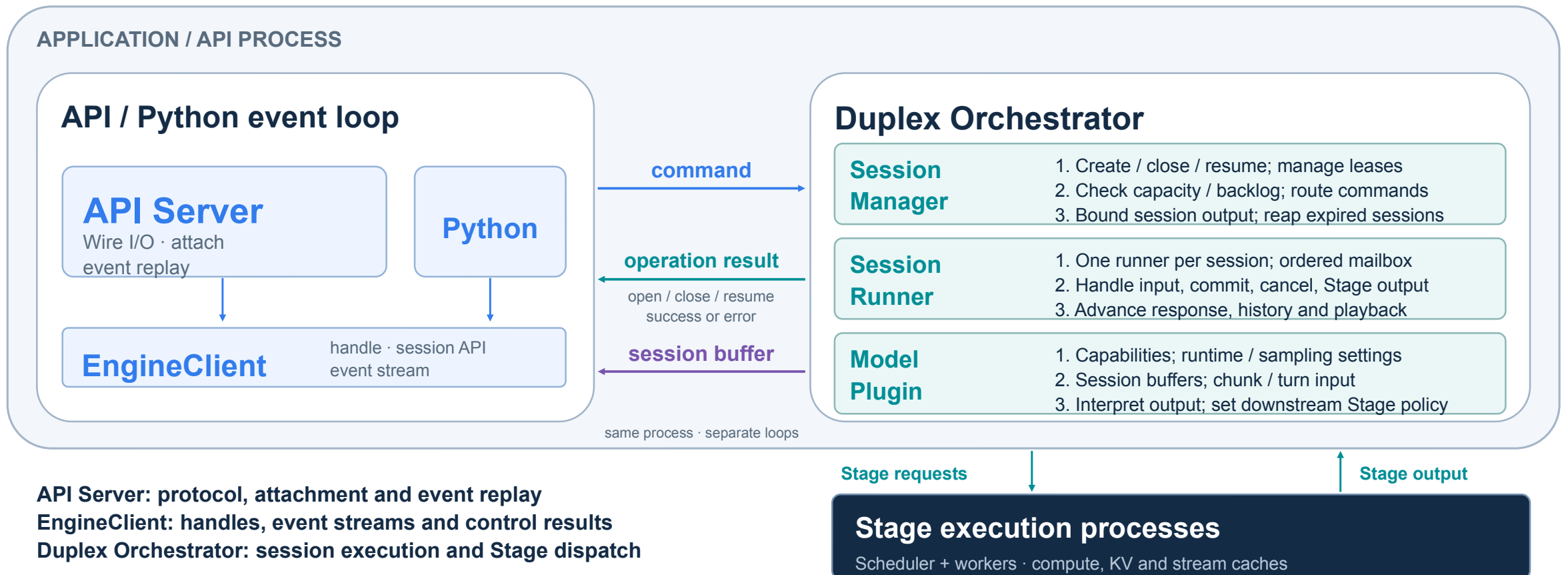
The interface carries the conversation; the execution mode determines how the model processes input.



Realtime also supports a separate turn-based path: /v1/realtime?duplex=0.

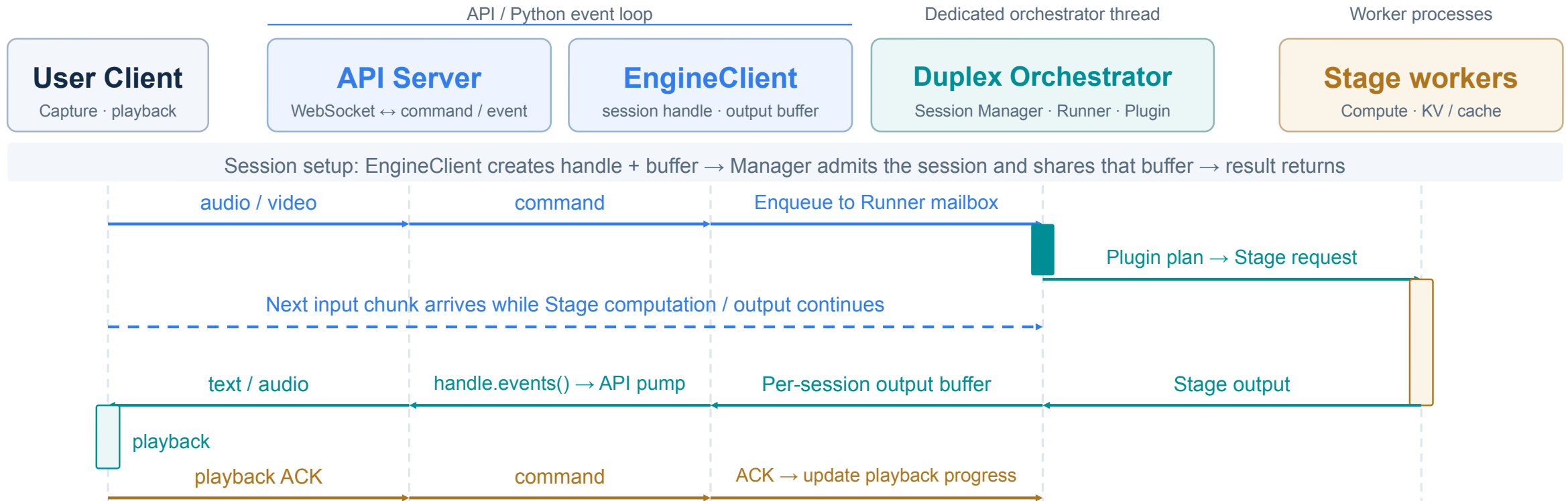
Unified duplex architecture, with clear roles at each layer

Duplex WebSocket and Python sessions share engine-side control, model plugins and bounded output delivery.



Full-duplex workflow

API Server handles the wire protocol; EngineClient exposes session handles; Runner / Plugin drives execution.



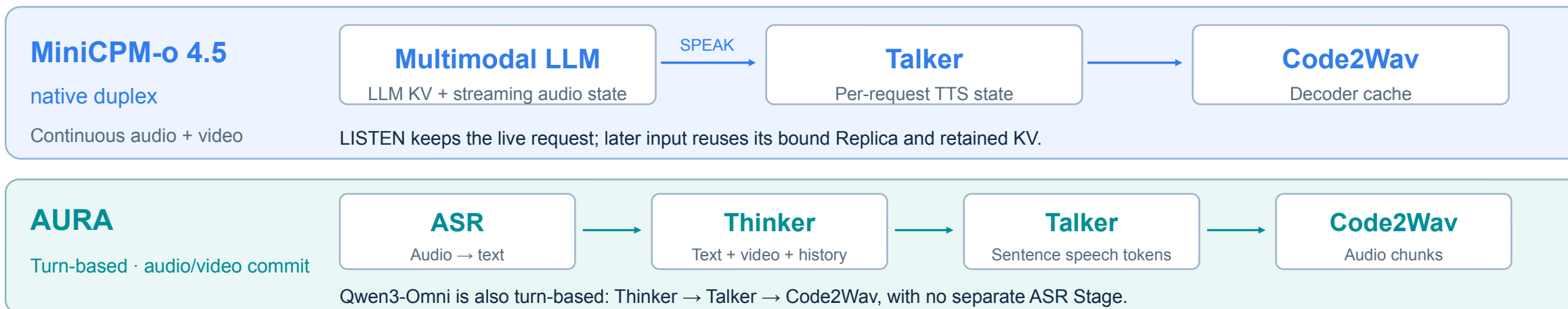
Continuous input · Bounded output per session · Playback progress reported separately

Session event ACKs stay at the API. Playback ACKs reach the Runner; history updates depend on model support.

State handling: live computation and conversation history

MiniCPM updates a live request; AURA and Qwen3-Omni submit a new request for each committed turn.

MODEL	INPUT HANDLING	STATE RETAINED ACROSS INPUTS
MiniCPM-o 4.5	Chunk → continue live request	LLM KV / audio state; configurable sliding window
AURA	Commit turn → new request	ASR text, video clips and assistant text; rebuild prompt each turn
Qwen3-Omni	Commit turn → new request	User audio, image frames and text; rebuild prompt each turn

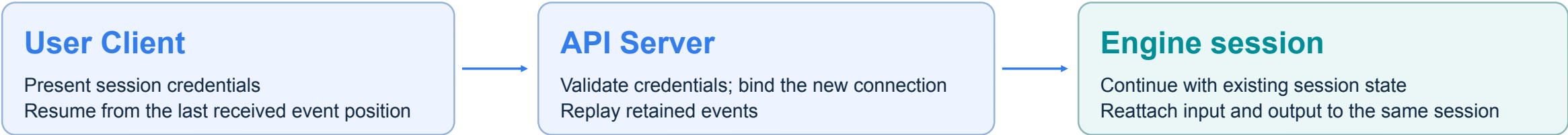


Stage = one model-compute step; Replica = an execution instance of that Stage.

MiniCPM state reuse requires a live request; AURA can continue sentence TTS within the same turn.

Reconnect and continue the same session

Reattach to a live engine session with its credentials and replay retained events from the last received position.



Requires the same live owner, a valid lease and retained events. Session resume does not provide process failover.

RESUME STEP	FRAMEWORK ACTION	USER CLIENT INTEGRATION
Session identity	Keep the same session	Python SDK retains credentials automatically; custom clients store session ID / resume token
Connection binding	Rebind the existing session	Reconnect the WebSocket and send session.resume
Event replay	Replay retained events	Provide the last received event position Receive retained events from the replay window

Live-session resume: MiniCPM · AURA · Qwen3-Omni · Nemotron. PersonaPlex requires a new session after disconnect.

Optimized streaming execution

Model-specific CUDA paths reduce repeated work across live input, generation and speech decoding.

01 MiniCPM: retain a bounded KV window

Basic mode: block-aligned sliding window
In-place Re-RoPE without copying retained KV

02 MiniCPM: audio / video CUDA graphs

Streaming audio and vision encoder CUDA graphs
Resident Code2Wav execution and fused kernels

03 PersonaPlex: batch live audio frames

Batch appends across active sessions
Replay Mimi encoder / decoder CUDA graphs

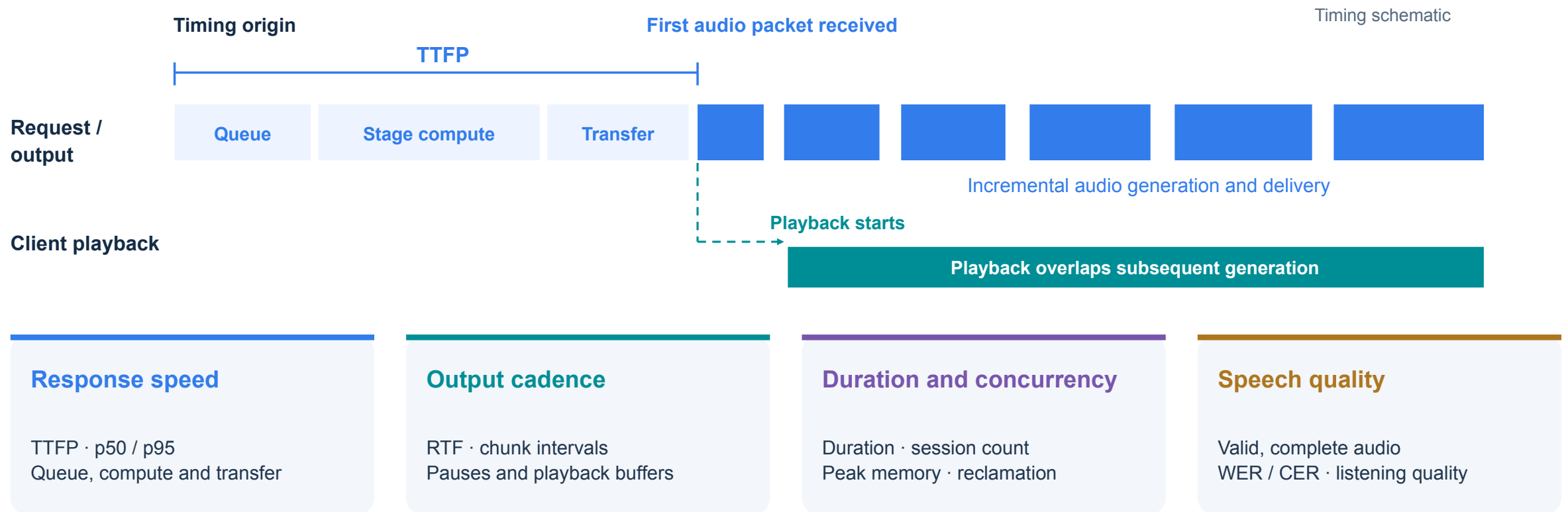
04 Qwen3-Omni: capture vision execution

Thinker image encoder CUDA graphs
Reuse captured execution for supported shapes

Select a matching CUDA profile; measure end-to-end TTFP and RTF at the target load.

From first audio to sustained conversation quality

OmniInteract and Omni-DuplexEval connect live session execution with timing and quality evaluation.



Fix model, hardware, concurrency and timing origin. Benchmark support and evaluation tasks are model-specific.

Performance at concurrency 1 and 4

Reported response latency and speech-generation efficiency at concurrency 1 and 4.

MiniCPM-o 4.5 · H100 · Seed-TTS text-to-speech

METRIC (MEAN)	CONCURRENCY 1	CONCURRENCY 4
TTFT ↓	161.8 ms	252.8 ms
TTFP ↓	635.3 ms	1451.9 ms
RTF ↓	0.27	0.64

AURA · 2×H200 · Omninteract realtime audio + video

METRIC (SPOKEN-TURN MEDIAN)	CONCURRENCY 1	CONCURRENCY 4
TTFT ↓	208 ms	244 ms
TTFP ↓	463.8 ms	480.1 ms

TTFT: time to first token. TTFP: time to first audio packet. RTF: generation time / audio duration. Lower is better.

MiniCPM: 2026-09-21 test records. AURA: 2 FPS, warmup excluded.

Tool calling across duplex models

Broader model coverage and consistent behavior during tool execution.

AVAILABLE TODAY

Qwen3-Omni Realtime

Tool parser in the turn-based Realtime path

Nemotron VoiceChat

Tool-result round trip in native duplex

UNIFIED DUPLEX ROADMAP

Model emits

Tool name + arguments



Application executes

External function



Result returns

Match the call_id



Model continues

Generate the reply

During execution

Define whether user input and speech output continue while the tool runs.

Turn safety

Handle interruption, timeouts, duplicate and late results;
keep old calls from affecting a new turn.

Framework + plugins

Framework owns shared call handling; plugins adapt model formats and behavior.
Extend coverage to Qwen3-Omni unified duplex, AURA and JoyAI.

Full-duplex roadmap

Near-term model integration, interaction and evaluation; longer-term session duration and serving scale.

NEAR TERM

JoyAI integration

Bring JoyAI into the unified duplex framework with model-specific input/output adapters

Unified tool calling

Shared call/result handling and model adapters
Interruption, timeout and stale-result handling

Semantic VAD

Turn completion based on meaning and context
Integration with commit and response policies

Broader realtime evaluation

Broader OmniInteract and Omni-DuplexEval coverage
More models, interaction tasks and regression checks

LONGER TERM

Longer sustained conversations

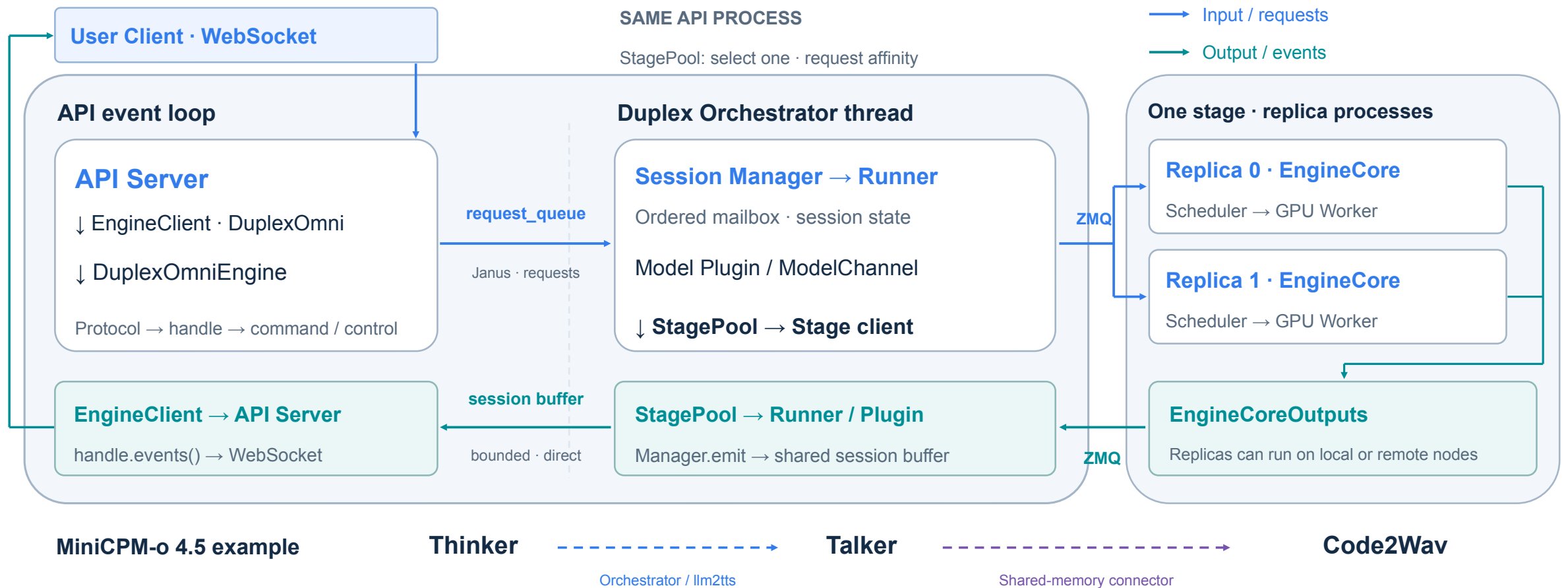
Bound context and history for extended audio/video
Maintain stable latency and memory as sessions grow

Scaled multi-replica serving

Higher session concurrency across Stage replicas
Session routing and ownership across API frontends

Duplex execution: input, model computation and output

Normal output uses a bounded per-session buffer shared by the API and Orchestrator threads.



Acoustic VAD vs. Semantic VAD

Often combined: acoustic VAD finds a pause, then a turn model checks whether the user has finished.

COMPARISON	Acoustic VAD	Semantic VAD PROPOSED EXTENSION
Decision signal	Speech activity + a configured silence duration	Meaning and context Audio models can also use prosody
Pause handling	May commit after enough silence, even if the sentence is unfinished	Can wait longer when the utterance appears incomplete
System action	Commit after the silence threshold. Start a response if enabled.	Commit when the turn is judged complete. Start a response if enabled.
Compute & placement	Low-latency CPU inference Current: CPU thread pool	Typically more compute per decision Planned: Omni Stage for heavier models

OPEN-SOURCE EXAMPLE: SILERO VAD + PIPECAT SMART TURN v3.2

“Book me a flight for tomorrow to ... [pause] ... Beijing.”

1. Silero: speech / non-speech and pauses

2. Smart Turn: complete / incomplete

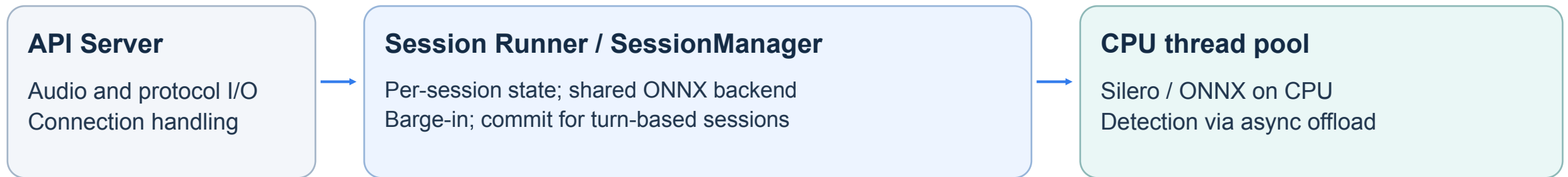
VAD can reduce noise-triggered events. Denoising and interruption intent detection are separate.

VAD: session policy with inference off the control loop

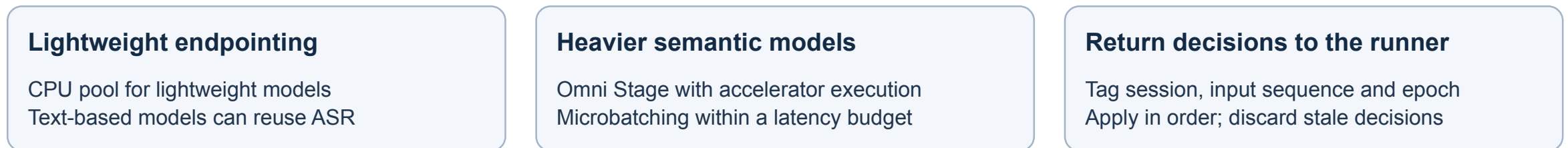
Server VAD uses Silero today; semantic endpointing can follow the same separation of responsibilities.

CURRENT SERVER VAD

Unified Duplex: same API process; control loop + CPU threads



SEMANTIC VAD · PROPOSED EXTENSION



Speech onset enables fast interruption; semantic endpointing detects turn completion, separately from LISTEN / SPEAK.

Disconnect, cancel and close have different effects

MiniCPM, AURA, Qwen3-Omni and Nemotron support live-session resume; PersonaPlex closes on disconnect.

TRIGGER	IN-FLIGHT WORK	SESSION / RESUME
Disconnect · resumable model	May keep generating; journal later events	Disconnect grace; resume while the session lives
response.cancel	Cancel response; invalidate its queued audio	Session stays open; cancelled work stays cancelled
session.close / lease expiry	Terminate and clean up associated work	Session ends; create a new session

SESSION LIFETIME
Resume within the disconnect grace
Session remains on its live owner

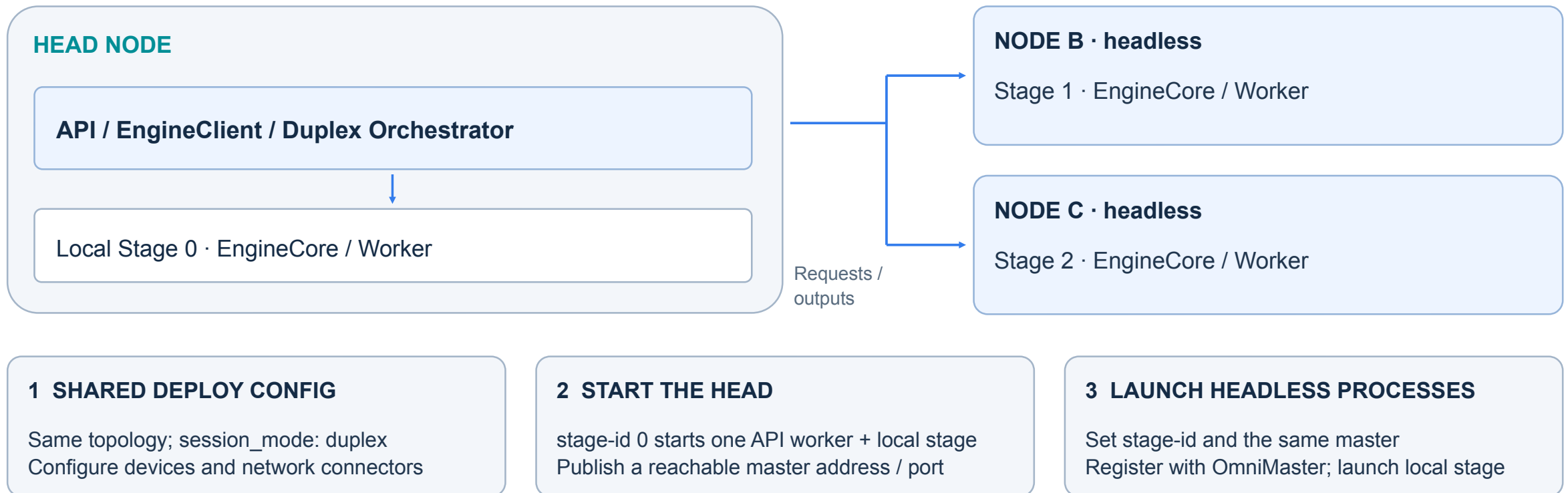
VALID CREDENTIALS
Valid resume token
Matching lease generation

REPLAY COVERAGE
Last received event position
Required events remain available

Defaults: disconnect grace 30s · idle TTL 300s · replay 60s / 8 MiB. Resume requires the same live owner.

Cross-node stages share the entrypoint Orchestrator

One API worker owns duplex sessions; the Head connects local and registered headless Stage processes.



Headless stages share the Head Orchestrator. Duplex rejects multiple API workers; cross-node data needs a network connector.

Code references

LAYER	IMPLEMENTATION ENTRY	READ FOR
API adapter	OmniDuplexSessionHandler	Wire events, attachment, replay
EngineClient	DuplexOmni / DuplexSessionHandle	Handles, shared output buffer, events
Transport	DuplexOmniEngine / DuplexOutputBuffer	Control queues; bounded session output
Session runtime	DuplexSessionManager / DuplexSessionRunner	Leases, state, mailbox, model policy
Stage orchestration	DuplexOrchestrator / StagePool	Initial / update, replica affinity
Model execution	Scheduler / Model Worker	Batching, compute, model caches